



Università degli Studi di Ferrara

UNIVERSITÀ DEGLI STUDI DI FERRARA
CORSO DI LAUREA IN INFORMATICA

Tesi Envass

Relatore:

Prof. Guido SCIAVICCO

Laureando:

Christian MICK

ANNO ACCADEMICO 2025 – 2026

Indice

	Page
1 Introduzione	5
2 Background	7
2.1 Evoluzione del Web e Tecnologie Client-Side	7
2.1.1 Il linguaggio di marcatura HTML	7
2.1.2 La formattazione tramite CSS	9
2.1.3 Framework per il Design Responsivo: Bootstrap	9
2.1.4 Interattività tramite JavaScript	10
2.2 Sviluppo Lato Server e Linguaggio PHP	12
2.2.1 Il Modello Architettuale Client-Server	12
2.2.2 Il Protocollo di Comunicazione HTTP	13
2.2.3 Evoluzione e Paradigmi del Linguaggio PHP	14
2.2.4 Gestione delle Dipendenze tramite Composer	16
2.3 Architettura del Framework Laravel	18
2.3.1 Il Pattern Architettuale MVC (Model-View-Controller): . .	18
2.3.2 Il Ciclo di Vita della Richiesta (Request Lifecycle):	18
2.3.3 Service Layer e Dependency Injection:	18
2.4 Persistenza dei Dati e Database Relazionali	19
2.4.1 Sistemi di Gestione di Database (RDBMS):	19
2.4.2 Eloquent ORM (Object-Relational Mapping):	19

2.4.3	Versionamento del Database: Migrations:	19
3	Progetto	21
4	Implementazione del progetto	23
5	Conclusione	25

Capitolo 1

Introduzione

Scrivi qua l' introduzione

Background

2.1 Evoluzione del Web e Tecnologie Client-Side

L'evoluzione del Web client-side si fonda sulla sinergia di tre tecnologie principali: HTML per la struttura, CSS per la presentazione e JavaScript per il comportamento.

2.1.1 Il linguaggio di marcatura HTML

Il Hypertext Markup Language (HTML) è il componente fondamentale del Web, definisce il linguaggio e la struttura dei contenuti web. Lo scopo del linguaggio HTML è quello di strutturare e definire semanticamente le informazioni. La sua evoluzione è contraddistinta da differenti fasi che corrispondono a versioni discrete dello stesso (come l'HTML4 o l'XHTML), ma ad oggi le specifiche HTML sono mantenute come uno "standard vivente" (Living Standard) mantenuto dal gruppo WHATWG (Web Hypertext Application Technology Working Group). La particolarità del modello prevede un aggiornamento continuo delle specifiche senza l'utilizzo di rilasci numerati. [1]

Alla base c'è il concetto di Hypertext (ipertesto) che definisce la struttura di link (collegamenti) permettendo la navigazione tra diverse pagine web o all'interno dello stesso documento. I link, a sua volta, rappresentano una componente essenziale della rete. L'architettura del World Wide Web (WWW) si fonda sulla pubblicazione di contenuti e sulla loro interconnessione tramite link. [2, cap. 5]

HTML non è un linguaggio di programmazione, è un linguaggio di marcatura. Utilizza una sintassi basata su tag (etichette racchiuse tra parentesi angolari `< >`) per istruire il browser su come interpretare e visualizzare i dati.[2, p. 32] Ad esempio, il tag `<h1>` definisce un'intestazione principale, il tag `<p>` delimita un paragrafo, e il tag `<a>` (chiamato "Anchor") è lo strumento tecnico che abilita la funzione di ipertesto citata, permettendo di collegare nodi informativi differenti all'interno del WWW.

Quando un browser riceve un documento HTML, ne genera una rappresentazione interna denominata Document Object Model (DOM). Il DOM è una struttura gerarchica ad albero che mappa tutti gli elementi della pagina, rendendoli accessibili e manipolabili tramite linguaggi di scripting.

Ogni documento segue una struttura ben definita in cui gli elementi possono essere annidati. I contenuti visibili all'utente sono racchiusi all'interno del tag `<body>`, mentre le informazioni di servizio, i metadati o i riferimenti a file esterni risiedono all'interno del tag `<head>`.

Infine, per differenziare e raggruppare elementi specifici, l'HTML fornisce attributi globali come `id` (per identificare in modo univoco un singolo elemento) e `class` (per classificare gruppi concettuali di elementi), che fungono da fondamentali punti di aggancio per stili e script.[2, ch. 5]

Gli elementi appena descritti permettono al HTML di definire lo scheletro informativo della pagina, ma delega gli aspetti visivi e comportamentali ad altri linguaggi specializzati.

2.1.2 La formattazione tramite CSS

I Cascading Style Sheets (CSS) rappresentano il linguaggio dedicato esclusivamente alla formattazione ed all'aspetto visivo dei documenti web. Se il HTML fornisce la semantica e i contenuti, il CSS interviene per garantire una rigorosa "separazione" tra i contenuti stessi e la loro presentazione esteriore. Questo paradigma consente il controllo centralizzato dell'aspetto visivo, permettendo di definire in modo totalmente indipendente e granulare dettagli cruciali come i colori, la tipografia, gli spazi (margini e padding) e la disposizione degli elementi sullo schermo.

Il linguaggio CSS si organizza attraverso un sistema di regole che collegano gli elementi di una pagina web al loro aspetto estetico. Ogni regola inizia con un selettore, ovvero il termine che indica al browser quali parti del testo o della pagina devono essere modificate. Dopo il selettore si trova la dichiarazione, scritta tra parentesi graffe, che contiene i dettagli pratici dello stile. Questa si divide a sua volta in una proprietà, che sceglie la caratteristica da cambiare come il colore o la grandezza, e in un valore, che stabilisce la modifica effettiva. Per rendere il lavoro più veloce e il codice più pulito, è possibile unire più selettori insieme in modo da assegnare lo stesso stile a diversi elementi contemporaneamente con un'unica istruzione.

Uno dei vantaggi più significativi di questa separazione è la possibilità di raccogliere tutte le regole all'interno di file esterni dotati di estensione .css. Un documento HTML può collegarsi a questi file esterni utilizzando il tag `<link>` inserito nel suo `<head>`. Questa tecnica consente a decine o centinaia di pagine web di condividere e puntare al medesimo foglio di stile; di conseguenza, una singola modifica apportata al file CSS si ripercuote istantaneamente sull'intero sito web, facilitando scalabilità, coerenza visiva ed operazioni di manutenzione.

2.1.3 Framework per il Design Responsivo: Bootstrap

L'ambiente in cui il Web viene fruito oggi è estremamente frammentato: i contenuti devono essere visualizzati su schermi di dimensioni enormemente diverse, dai grandi monitor desktop ai tablet, fino ai piccoli display degli smartphone, non-

ché su dispositivi assistivi o degli schermi televisivi. Questa variabilità ha reso imprescindibile l'adozione del "Design Responsivo" e di filosofie progettuali come il "Mobile First", introdotto da esperti come Luke Wroblewski, che spingono per ottimizzare i siti web primariamente per i dispositivi mobili prima di adattarli agli schermi più ampi.

Costruire layout moderni, complessi e visivamente coerenti partendo da zero richiede un notevole e ripetitivo sforzo di programmazione CSS. Per standardizzare e velocizzare drasticamente questo processo, l'industria fa largo uso di "framework", ovvero insiemi organizzati di librerie, convenzioni e codici collaudati che forniscono solide fondamenta applicative. Questi framework integrano nativamente sistemi a griglia (Grid system) e regole flessibili (Media Queries) che riorganizzano dinamicamente i blocchi di contenuto in base alla larghezza e all'orientamento dello schermo (Viewport) del dispositivo.

Uno dei loro compiti principali è risolvere il problema della frammentazione tra i vari browser, poiché ogni programma interpreta gli standard del codice in modo differente. Strumenti come Bootstrap intervengono applicando tecniche di normalizzazione o reset, che servono a livellare le differenze estetiche di base e a garantire che elementi come pulsanti, margini e font appaiano identici ovunque. Oltre alla coerenza visiva, questi sistemi favoriscono l'inclusività integrando direttamente nel codice il supporto agli standard di accessibilità, come i ruoli ARIA che permettono alle interfacce di essere facilmente utilizzabili anche da persone con disabilità attraverso tecnologie assistive.

In questo panorama, Bootstrap si posiziona come uno dei framework frontend più potenti, completi e diffusi al mondo, concepito per consentire uno sviluppo rapidissimo.

2.1.4 Interattività tramite JavaScript

Se il HTML è la struttura e il CSS è la presentazione, JavaScript è il motore comportamentale del Web. JavaScript è un linguaggio di programmazione ad alto livello, solitamente compilato Just-In-Time (JIT) dinamico e multi-paradigma (supportando stili imperativi, funzionali e orientati agli oggetti).

Sebbene nato come linguaggio di scripting elementare, si è evoluto nello standard

ECMAScript, introducendo con la versione ES6 costrutti avanzati come classi e moduli. I programmi JavaScript non vengono eseguiti nel vuoto, bensì necessitano di un "ambiente host" (host environment) che conferisca loro le facoltà per interfacciarsi col mondo esterno. Nel dominio dello sviluppo frontend, l'ambiente host è il browser web stesso. Il browser dota JavaScript di svariate API (Application Programming Interface, interfacce di programmazione esposte all'applicazione). La più importante è il DOM, attraverso la quale il linguaggio può ricercare elementi, mutare l'albero dei nodi, inserire o rimuovere tag e modificarne i contenuti. JavaScript opera seguendo un'architettura guidata dagli eventi e di tipo asincrono, il che significa che il codice non occupa costantemente le risorse del computer, ma resta in attesa di segnali specifici. Questi segnali, chiamati eventi, possono essere azioni compiute dall'utente, come un clic o la pressione di un tasto, oppure risposte inviate da un server. Per rispondere a questi stimoli, gli sviluppatori utilizzano le funzioni di callback, ovvero blocchi di istruzioni che vengono attivati dal browser solo nel momento in cui si verifica l'evento a cui sono stati associati. Questo meccanismo, gestito da un sistema chiamato event loop, permette alle pagine web di rimanere fluide e reattive, poiché il programma non si blocca mai nell'attesa di un comando, ma esegue le operazioni solo quando è realmente necessario.

2.2 Sviluppo Lato Server e Linguaggio PHP

2.2.1 Il Modello Architetture Client-Server

Il modello di architettura client-server è uno dei due principali paradigmi architetturali del web. In questo modello l'infrastruttura è suddivisa tra due processi comunicanti: un host "always-on" (il server), che possiede un indirizzo IP fisso e noto, e i dispositivi degli utenti (i client), che inviano richieste al sistema centrale. Nel contesto dello sviluppo web, questa separazione si traduce nella distinzione tra "frontend" (il lato client) e "backend" (il lato server).[\[3, ch. 2.1\]](#)

Lo scopo del server è gestire richieste provenienti da moltitudini di host client. I client per contro non comunicano mai direttamente tra di loro nell'architettura Client-Server.

I client sono costituiti dai browser (user-agents) in esecuzione sui dispositivi degli utenti. Le applicazioni client-side elaborano l'interfaccia utente utilizzando tecnologie come HTML, CSS e JavaScript. Il web presenta un limite strutturale, il browser dell'utente per la natura stessa del web, è un ambiente insicuro. Chiunque abbia un minimo di competenza può aprire gli strumenti di sviluppo, disponibili su ogni browser e leggere e modificare il codice. [\[2, ch. 2\]](#)

Il server, quindi, deve agire come garante dell'integrità e della logica del sistema tramite lo scripting server-side. Lo sviluppo backend si concentra interamente sui software e sui database in esecuzione sul server. Esso deve gestire ogni singola operazione, validazione di input, interagire con il database e gestire la memorizzazione dei file.[\[2, ch. 1\]](#)

Poiché il client è un ambiente esposto alle azioni dell'utente, l'unico modo per assicurare la validità dei dati è demandare al server ogni operazione di verifica. Ogni transazione, sia essa la visualizzazione di un documento riservato o il caricamento di un nuovo file, viene validata dalla logica di business residente sul server. Se l'utente non possiede i privilegi necessari o se i dati inviati non superano i rigorosi controlli imposti dalle procedure backend, la transazione viene bloccata alla radice. In questo ecosistema, il client funge esclusivamente da mezzo di visualizzazione e di raccolta degli input, mentre il sistema centrale detiene l'autorità assoluta e irrevocabile sulle operazioni e sui registri di audit.

2.2.2 Il Protocollo di Comunicazione HTTP

L'Hypertext Transfer Protocol (HTTP) è il protocollo fondamentale del web usato per la trasmissione di informazioni su internet. È composto da due programmi, quello per il server e quello per il client. I due programmi su terminali diversi comunicano scambiandosi messaggi HTTP. Il protocollo HTTP definisce la struttura di questi messaggi e le modalità con cui il client e il server li scambiano.

Il ciclo vitale di questa comunicazione è basato su un rigido modello Request-Response. Il processo ha inizio quando il browser dell'utente necessita di un'informazione o deve inviare dei dati. Il client invia un messaggio di richiesta HTTP (HTTP request) al server. Questo messaggio contiene campi di intestazione (header) e un metodo (come GET per richiedere dati, POST per inviare, DELETE per cancellare, etc.).

Il server è costantemente in ascolto su una porta specifica (solitamente la 80 per HTTP e la 443 per HTTPS) e una volta che il server ha ricevuto questa richiesta, poi recupera, assembla e invia le informazioni sotto forma di messaggio di risposta HTTP (HTTP response) il più rapidamente possibile. Questa risposta contiene un codice di stato (che indica il successo o il fallimento dell'operazione, come "200 OK" o "404 Not Found") e nel corpo del messaggio, i dati o la pagina elaborata da restituire all'utente.

L'HTTP è un protocollo stateless (senza stato), quindi ogni transazione è indipendente dalle precedenti. Il server elabora la singola richiesta e invia i file al client senza conservare memoria delle transazioni precedenti. Ogni ciclo di Request-Response è un evento isolato, indipendente e autoconclusivo. Per esempio, se un utente richiede lo stesso documento due volte in pochi secondi, il server lo elaborerà nuovamente da zero, non avendo memoria dell'azione appena compiuta. Per mantenere la persistenza delle informazioni (come le sessioni di login), si utilizzano meccanismi esterni come i cookie.

Per introdurre una memoria storica all'interno del sistema (indispensabile, ad esempio, per tracciare le sessioni utente autenticate in un sistema documentale), le applicazioni non si affidano a HTTP in sé, ma costruiscono un livello di sessione utente al di sopra di HTTP tramite l'utilizzo dei cookie. Questo consente al server di identificare le azioni del client e mantenere registri di audit sicuri per ogni singola transazione effettuata.

Posizione	Linguaggio di Programmazione	Percentuale (%)
1	PHP	71,4%
2	Ruby	6,7%
3	JavaScript (Node.js)	6,2%
4	Java	5,4%
5	Scala	5,0%
6	ASP.NET	4,4%
7	Static files	2,1%
8	Python	1,2%
9	ColdFusion	0,2%
10	Perl	0,1%

Tabella 2.1: Utilizzo dei linguaggi di programmazione lato server [4].

HTTP è anche affidabile, utilizza TCP come protocollo di trasporto sottostante. TCP fornisce un servizio di trasferimento dati affidabile, garantendo allo sviluppatore che i messaggi di richiesta e i file di risposta arrivino intatti e senza errori a destinazione[3, ch. 2.2]

2.2.3 Evoluzione e Paradigmi del Linguaggio PHP

Il linguaggio PHP, acronimo ricorsivo per *Hypertext Preprocessor* è uno dei linguaggi di *scripting server-side* di riferimento per lo sviluppo *web*. Come mostrato nella Tabella 2.1, PHP è la tecnologia sottostante al 71,4% dei sistemi *web*, che dispongono di un *back-end*. A titolo di confronto, il suo concorrente più vicino (Ruby) detiene solo il 6,7% del mercato. Questa diffusione capillare si traduce in un ecosistema maturo e ampiamente supportato, esistono moltissime librerie testate (tramite strumenti come *Composer*), documentazioni, libri, *tutorial*, *framework* basati su questo linguaggio e alla garanzia di un supporto continuo a lungo termine.

La scelta di PHP come linguaggio *back-end* garantisce l'utilizzo di una tecnologia ampiamente collaudata su scala globale e supportata da un'infrastruttura

server massiva. Il linguaggio inoltre offre un'integrazione nativa con principali sistemi di gestione di database relazionali (RDBMS).

L'attuale livello di maturità architetturale è il risultato di una profonda e stratificata evoluzione tecnica. PHP nasce e si sviluppa con il preciso e unico intento di essere un linguaggio per lo *scripting* lato *server*. Viene concepito originariamente nel 1994 da Rasmus Lerdorf come un semplice insieme di file binari CGI (*Common Gateway Interface*) scritti in linguaggio C e denominati *Personal Home Page Tools*. Il primo punto di svolta avvenne con il rilascio della versione PHP 3.0 nel 1998, derivata da una riscrittura completa del *parser* interno ad opera di Andi Gutmans e Zeev Suraski, dove per la prima volta introdussero il supporto, ancora embrionale, alla programmazione orientata agli oggetti (OOP) e permise l'integrazione di numerosi database in modo nativo. Lo sviluppo delle prestazioni e della modularità è proseguito con l'introduzione dello *Zend Engine* in PHP 4.0, ottimizzando drasticamente le prestazioni del codice e la modularità per applicazioni *enterprise*, e si è consolidato con PHP 5, che ha offerto un modello a oggetti completamente rinnovato e molto più robusto. Le iterazioni più recenti, segnatamente PHP 7 e PHP 8, hanno ulteriormente consolidato il paradigma OOP introducendo l'uso di tipizzazioni strette (*union types*, *named arguments*), la compilazione *Just-In-Time* (JIT) e una gestione delle risorse di memoria nettamente ottimizzata.[5]

Un fattore chiave in questa maturazione è l'evoluzione verso il *Gradual Typing*. Storicamente nato come linguaggio a tipizzazione debole e dinamica, PHP ha implementato un sistema di controllo dei tipi progressivo e rigido. L'introduzione degli *Union* e *Intersection Types*, combinata con la direttiva di tipizzazione rigida, permette una precisa definizione dei contratti del codice. Ciò agevola gli strumenti di analisi statica nel rilevare anomalie e incongruenze già in fase di sviluppo.[6]

Il linguaggio di programmazione PHP si è strutturato come un ecosistema multiparadigma. I paradigmi principali supportati e comunemente utilizzati nello sviluppo moderno sono tre:[6]

- **Paradigma Orientato agli Oggetti (OOP):** Rappresenta il paradigma dominante utilizzato dai *framework* moderni come Laravel. Il linguaggio supporta l'incapsulamento rigoroso tramite modificatori di visibilità, l'ereditarietà singola, le interfacce, le classi astratte e il polimorfismo.

- **Paradigma Procedurale:** Costituisce la base storica del linguaggio e l'architettura nativa di molte delle sue funzioni principali. Si basa su una struttura lineare di funzioni e costrutti di controllo imperativi, questo modello è particolarmente indicato per l'implementazione di *script* d'automazione e componenti *software* a bassa complessità.
- **Supporto al Paradigma Funzionale:** Sebbene non sia un linguaggio funzionale nativo, PHP implementa funzioni di prima classe, funzioni anonime (*Closure*) e funzioni freccia (*Arrow Functions*). Inoltre, sono stati implementati costrutti nativi di ordine superiore come `array_map()` e `array_filter()`, che consentono l'elaborazione dei dati secondo logiche dichiarative e non imperative.

2.2.4 Gestione delle Dipendenze tramite Composer

Composer [7] è uno strumento di gestione delle dipendenze in *PHP*. A differenza dei tradizionali gestori di pacchetti, come *Yum* o *Apt*, *composer* gestisce i pacchetti/librerie su base per progetto e le installa in una *directory* locale (solitamente denominata *vendor*). Tale architettura, concettualmente ispirata a strumenti quali *npm* o *bundler*, permette di dichiarare, gestire e installare le librerie software esterne (pacchetti) richieste da un progetto, risolvendo automaticamente le relazioni di interdipendenza tra di esse.

Composer è formato da tre elementi fondamentali:

- **composer.json:** *File* di configurazione in formato *json* al nucleo del progetto. Contiene i metadati dell'applicazione e l'elenco delle dipendenze dirette per lo sviluppo (*require-dev*) e per la produzione (*require*).
- **composer.lock:** *File* generato automaticamente dopo la prima installazione o l'aggiornamento dei pacchetti. Registra le versioni esatte di ciascun pacchetto, così da poter garantire *build* riproducibili e deterministiche.
- **Packagist:** Il *repository* pubblico principale e predefinito di *Composer*. Aggrega i pacchetti *open source PHP* disponibili globalmente e viene interrogato da *Composer* per localizzare i pacchetti da scaricare.

L'algoritmo di risoluzione individua le dipendenze corrette interrogando *Packagist* per raccogliere i metadati dei pacchetti richiesti. Successivamente, analizza i *tag* e i rami dei sistemi di controllo di versione (VCS) per valutare i vincoli di versione e calcolare la combinazione di librerie compatibile con le richieste del software. Infine, il processo termina definendo le versioni esatte di ciascun pacchetto che verranno poi cristallizzate all'interno del file `composer.lock`.

Composer dispone inoltre di un *autoloader*, che è un meccanismo che automatizza il caricamento dei file *PHP* all'interno del progetto, eliminando la necessità di inserire manualmente istruzioni come *require* o *include* in ogni *script*. Adottando standard rigorosi di interoperabilità come PSR-4, il sistema di *autoloading* mappa in modo trasparente i *namespace* (spazi dei nomi) logici delle classi sui percorsi fisici del *filesystem*.

L'inclusione dell'*Autoloader* all'interno del progetto consente il caricamento e l'utilizzo dinamico delle classi a *runtime*, nel momento esatto in cui vengono istanziate. Questo processo si chiama *Lazy loading*, quindi quando il programma cerca di istanziare o utilizzare una classe non caricata in memoria, l'*Autoloader* intercetta la richiesta, controlla la mappa precedentemente generata e include dinamicamente il file richiesto.

2.3 Architettura del Framework Laravel

Il nucleo centrale del background. Qui si approfondisce la teoria del framework.

2.3.1 Il Pattern Architeturale MVC (Model-View-Controller):

Model:

Rappresentazione dei dati e logica di business.

View:

Il sistema di templating (Blade) e la presentazione.

Controller:

Intermediazione e gestione del flusso applicativo.

2.3.2 Il Ciclo di Vita della Richiesta (Request Lifecycle):

Routing:

Definizione degli endpoint e smistamento del traffico.

Middleware:

Spiegazione dei filtri di sicurezza e autenticazione.

2.3.3 Service Layer e Dependency Injection:

Approfondimento su come disaccoppiare la logica dai controller (essenziale per una tesi di alto livello).

2.4 Persistenza dei Dati e Database Relazionali

Spiegazione di come le informazioni vengono salvate e recuperate.

2.4.1 Sistemi di Gestione di Database (RDBMS):

Teoria dei database relazionali e linguaggio SQL.

2.4.2 Eloquent ORM (Object-Relational Mapping):

Come Laravel trasforma le tabelle del database in oggetti manipolabili.

Relazioni tra Entità:

One-to-One, One-to-Many, Many-to-Many.

2.4.3 Versionamento del Database: Migrations:

Concetto di evoluzione controllata dello schema senza intervento manuale su SQL.

Progetto

Implementazione del progetto

Conclusione

Bibliografia

- [1] *HTML Standard*. URL: <https://html.spec.whatwg.org/multipage/introduction.html#introduction> (visitato il giorno 09/04/2026).
- [2] Jennifer Niederst Robbins. *Learning Web Design: A Beginner's Guide to HTML, CSS, Javascript, and Web Images*. Sixth edition. Santa Rosa, CA: O'Reilly, 2025. 1 p. ISBN: 978-1-0981-3768-7.
- [3] James F. Kurose e Keith W. Ross. *Computer Networking: A Top-down Approach*. Eighth edition. Hoboken: Pearson, 2021. ISBN: 978-0-13-668155-7.
- [4] *Usage Statistics and Market Share of Server-side Programming Languages for Websites, May 2026*. URL: https://w3techs.com/technologies/overview/programming_language (visitato il giorno 08/05/2026).
- [5] *PHP: History of PHP - Manual*. URL: <https://www.php.net/manual/en/history.php.php> (visitato il giorno 08/05/2026).
- [6] *PHP: Preface - Manual*. URL: <https://www.php.net/manual/en/index.php> (visitato il giorno 08/05/2026).
- [7] *Composer*. URL: <https://getcomposer.org/doc/> (visitato il giorno 08/05/2026).